



Rust

Cristiano Damiani Vasconcellos
cristiano.vasconcellos@udesc.br

Traits

Rust suporta o polimorfismo por meio de duas características: generics e traits. Traits são inspiradas em classes de tipos de Haskell, mas também podem ser comparadas com interfaces ou classes abstratas de linguagens orientadas a objetos, por exemplo:

```
pub trait Display {  
    fn fmt(&self, f: &mut Formatter<'_>) -> Result<(), Error>;  
}
```

Trait: Eq

`x.eq(y)` também pode ser escrito como `x == y` e `x.ne(y)` pode ser escrito como `x != y`.

```
pub trait PartialEq<Rhs = Self>
where
    Rhs: ?Sized,
{
    fn eq(&self, other: &Rhs) -> bool;

    fn ne(&self, other: &Rhs) -> bool { ... }
}

pub trait Eq: PartialEq { }
```

Trait: Display

```
#[derive(Debug)]
struct Ponto {
    x:i32, y:i32
}

fn main() {
    let p1 = Ponto{x:1, y:10};
    println!("{p1:?}");
}
```

Trait: Display

```
#[derive(Debug)]
struct Ponto {
    x:i32, y:i32
}

impl fmt::Display for Ponto {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        write!(f, "({},{})", self.x, self.y)
    }
}

fn main() {
    let p1 = Ponto{x:1, y:10};
    println!("{p1:?}");
    println!("{p1}");
}
```

Trait: Display

```
enum Semana{
    Dom, Seg, Ter, Qua, Qui, Sex, Sab
}

use Semana::*;

impl fmt::Display for Semana {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        let dia = match *self {
            Dom => "Domingo",
            Seg => "Segunda",
            Ter => "Terça",
            Qua => "Quarta",
            Qui => "Quinta",
            Sex => "Sexta",
            Sab => "Sábado"
        };
        write!(f, "{dia}")
    }
}
```

Trait: PartialEq

```
enum Semana{
    Dom, Seg, Ter, Qua, Qui, Sex, Sab
}

use Semana::*;

impl PartialEq for Semana{
    fn eq (&self, r:&Self) -> bool {
        match *self {
            Dom => match *r {Dom => true, _ => false},
            Seg => match *r {Seg => true, _ => false},
            Ter => match *r {Ter => true, _ => false},
            Qua => match *r {Qua => true, _ => false},
            Qui => match *r {Qui => true, _ => false},
            Sex => match *r {Sex => true, _ => false},
            Sab => match *r {Sab => true, _ => false}
        }
    }
}
```

Trait: PartialOrd

```
pub trait PartialOrd<Rhs = Self>: PartialEq<Rhs>
where
    Rhs: ?Sized,
{
    fn partial_cmp(&self, other: &Rhs) -> Option<Ordering>;

    fn lt(&self, other: &Rhs) -> bool { ... }
    fn le(&self, other: &Rhs) -> bool { ... }
    fn gt(&self, other: &Rhs) -> bool { ... }
    fn ge(&self, other: &Rhs) -> bool { ... }
}

pub enum Ordering {
    Less = -1,
    Equal = 0,
    Greater = 1,
}
```


Trait: Ord

```
pub trait Ord: Eq + PartialOrd {  
    fn cmp(&self, other: &Self) -> Ordering;  
  
    fn max(self, other: Self) -> Self  
        where Self: Sized { ... }  
    fn min(self, other: Self) -> Self  
        where Self: Sized { ... }  
    fn clamp(self, min: Self, max: Self) -> Self  
        where Self: Sized { ... }  
}
```